S&P; 500 Derin Öğrenme Modeli Hiperparametre Denemeleri ve Sonuç Analizi

T+5 Gün Fiyat Yönü Tahmini — İkili Sınıflandırma

Konu	Makro olay etrafında S&P 500 hisse fiyat yönü tahm
Model Türü	Feedforward (Dense) Yapay Sinir Ağı
Problem Tipi	İkili Sınıflandırma (Yükseliş / Düşüş)
Kullanılan Framework	TensorFlow / Keras
Toplam Deney Sayısı	6 farklı konfigürasyon
Test Edilen Veri Seti Sayısı	2 (236k satır ve 17.5k satır)
En İyi Sonuç	Test Acc: %56.77 (17.5k satır, temel mimari)

1. Giriş ve Problem Tanımı

Bu çalışmada, S&P; 500 endeksiyle ilişkili hisse senetlerinin ve emtiaların makro olaylar (ABD Seçimleri, TÜFE açıklamaları, FED kararları vb.) etrafındaki fiyat hareketleri üzerinden 5 gün sonraki fiyat yönünün (yükseliş/düşüş) tahmin edilmesi amaçlanmıştır. Problem bir ikili sınıflandırma (binary classification) problemidir.

Çalışmanın asıl amacı, farklı hiperparametre kombinasyonları ile model davranışının nasıl değiştiğini gözlemlemek ve gerçek hayattaki finansal veri için en uygun konfigürasyonu bulmaktır. Toplam 6 farklı deneme yapılmıştır.

Temel Model Mimarisi (Baseline)

Tüm denemelerde temel yapı olarak küçük ve kompakt bir Feedforward ağ kullanılmıştır. Aşağıdaki mimari hem başlangıç noktası hem de daha sonra karşılaştırma referansı olmuştur:

Katman	Açıklama
Giriş	9 özellik (veya yeni veri setinde 7 özellik)
Dense (64)	Gizli katman, ReLU aktivasyonu
Dropout (0.5)	Aşırı öğrenmeyi (overfitting) önler
Dense (32)	Gizli katman, ReLU aktivasyonu
Dropout (0.5)	İkinci dropout katmanı
Dense (1)	Çıkış katmanı, Sigmoid aktivasyonu (0-1 olasılık)

Toplam parametre sayısı: 2.753 (9 özellik için) / 2.625 (7 özellik için). Bu oldukça kompakt bir modeldir — finansal verinin gürültülü yapısında büyük modeller ezber yapmaya eğilimli olduğu için küçük mimari tercih edilmiştir.

Kullanılan Özellikler

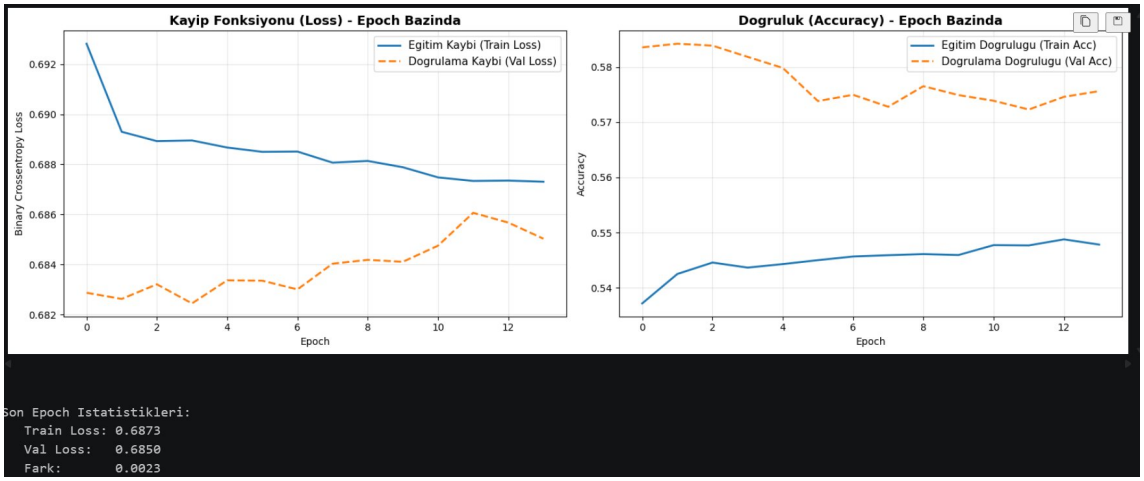
Veri Seti	Kullanılan Özellikler
236k satırlık veri	Log_Getiri, Volatilite_10g, Volatilite_30g, RSI_14, MACD_12_26_9, MACDh_12_26_9, MACDs_12_26_9, BBL
17.5k satırlık veri	Log_Getiri, Volatilite_10g, Volatilite_30g, Log_Hacim, Hacim_Degisimi, RSI_14, SMA_Uzakl

2. Deneme 1 — İlk Baseline (Adam Optimizer)

Kullanılan Ayarlar

Parametre	Değer
Optimizer	Adam (varsayılan learning_rate=0.001)
Batch Size	64
Epoch (maksimum)	50
Patience (EarlyStopping)	10
Aktivasyon	ReLU (gizli) / Sigmoid (çıkış)
Dropout	0.5
Veri Seti	236.000 satırlık veri

Eğitim Grafiği



Eğitim ve doğrulama kayıp/doğruluk eğrileri. Eğitim 14. epoch'ta EarlyStopping ile durmuştur.

Sonuçlar

Metrik	Değer
Train Loss	0.6873
Val Loss	0.6850
Train-Val Fark	0.0023
Test Accuracy	Test ayrıca yapılmadı (ilk deneme)

Yorumlama

Bu deneme nasıl gitti? Başlangıç için gayet başarılı. Eğitim ve doğrulama kayıpları (Train ve Val Loss) birbirine çok yakın seyrediyor. Aralarındaki fark sadece 0.0023 — bu, modelin ezberlemediği, iyi bir şekilde genelleme yaptığı anlamına gelir.

İlginç gözlem: Grafikte val loss (turuncu kesikli çizgi) train loss'un (mavi) altında kalıyor. Normalde tam tersi olur. Bunun tek sebebi: Dropout(0.5) çok agresif. Eğitim sırasında nöronların yarısı rastgele kapatılıyor, değerlendirme sırasında hepsi açık. Bu yüzden

'sahte' olarak val loss düşük görünüyor. Bu, aslında hafif bir underfitting işaretidir — model veriden yeterince bilgi çıkaramamış olabilir.

Durum: Sağlıklı bir başlangıç. Baseline olarak kullanılmaya uygundur.

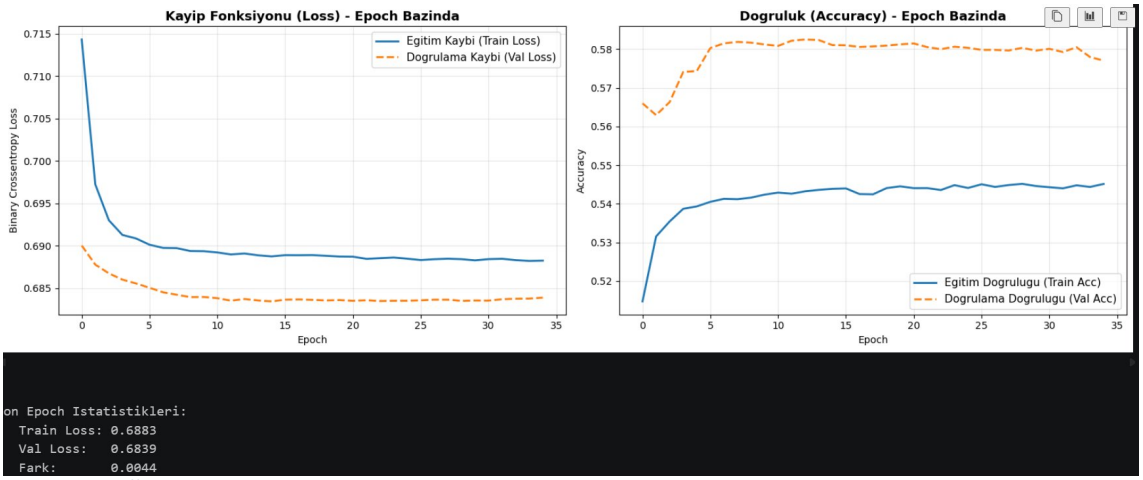
3. Deneme 2 — Learning Rate Düşürüldü

Kullanılan Ayarlar

Deneme 1'deki model biraz yavaş öğrendiği için öğrenme oranı düşürüldü ve daha uzun eğitilmesi için epoch/patience/batch değerleri arttırıldı.

Parametre	Değer (değişiklik)
Optimizer	Adam (learning_rate=0.0001) — 10 kat düşürüldü
Batch Size	128 (64'ten arttırıldı)
Epoch (maksimum)	150 (50'den arttırıldı)
Patience	20 (10'dan arttırıldı)
Diğer	Mimari, dropout ve aktivasyon aynı

Eğitim Grafiği



Eğitim 35. epoch'ta durmuş. Son epoch: Train Loss 0.6883, Val Loss 0.6839.

Sonuçlar

Metrik	Değer
Train Loss	0.6883
Val Loss	0.6839
Train-Val Fark	0.0044
Test Loss	0.6865
Test Accuracy	%55.87
Eğitim Epoch	35

Yorumlama

Grafiğin karakteri: İlk 5-10 epoch hızlı bir düşüş oluyor, sonra her iki eğri de düz bir şekilde (plato) ilerliyor. Yani model ilk birkaç epoch'ta ne öğrenecekse öğreniyor, sonrası ise düz çizgi. Bu, 'modelin yapabileceğinin sınırına erken ulaştığını' gösterir.

Deneme 1'e göre kıyas: Learning rate düşürülünce eğriler daha yumuşak ve stabil oldu. Train-Val farkı 0.0023'ten 0.0044'e çıktı — bu 'kötüleşme' değil, aslında daha gerçekçi bir tablo. Test doğruluğu %55.87 — rastgele tahminden (%54.3 yükseliş baseline) sadece %1.5 daha iyi, yani modelin çıkardığı gerçek sinyal çok küçük.

Durum: Başarılı bir stabilizasyon. Ancak accuracy kayda değer artmadı — verinin sinyal sınırına yaklaşıyoruz.

4. Denemeler 3 ve 4 — AdamW Optimizer

Adam optimizer'ın iyileştirilmiş versiyonu olan AdamW (weight decay ayrılmış versiyon) denendi. Bu optimizer ağırlıkları daha iyi düzenler (regularize eder). İki farklı ayarla çalıştırıldı:

Deneme 3 — Varsayılan AdamW

Parametre	Değer
Optimizer	AdamW (varsayılan ayarlarla)
Batch Size / Epoch	Deneme 2 ile aynı
Test Loss	0.6868
Test Accuracy	%55.93
Eğitim Epoch	22 (erken durdu)

Deneme 4 — Düşük LR + Yüksek Weight Decay + Uzun Eğitim

Parametre	Değer
Optimizer	AdamW(learning_rate=0.0001, weight_decay=0.004)
Batch Size	512
Epoch (maksimum)	750
Patience	75
Train Loss	0.6877
Val Loss	0.6838
Train-Val Fark	0.0039
Test Loss	0.6867
Test Accuracy	%55.77
Eğitim Epoch	121

Yorumlama

Bu iki deneme nasıl gitti? Test accuracy değerlerine bakalım:

Deneme	Test Acc	Fark (baseline)
Deneme 2 (Adam)	%55.87	—
Deneme 3 (AdamW)	%55.93	+%0.06
Deneme 4 (AdamW + yüksek WD)	%55.77	-%0.10

Öğrendiğimiz ders: Optimizer değiştirmek (Adam → AdamW) veya ayarlarıyla oynamak sonuçlarda çok küçük değişiklikler yarattı. Değişim %0.1'in altında. Bu değer istatistiksel olarak gürültü seviyesindedir — yani aslında anlamlı bir fark yok. Aynı modeli farklı rastgele tohumla 5 kez çalıştırsanız, aradaki fark bu kadar olurdu.

Deneme 4'te dikkat çeken: 121 epoch boyunca eğitildi (çok uzun), ama sonuç baseline ile neredeyse aynı. Bu bize şunu söyler: veri setinin sinyal tavanı düşük. Ne kadar uzun veya ne kadar farklı optimizer ile eğitirseniz eğitin, %56 duvarına çarpıyorsunuz.

Durum: Optimizer değişikliği tek başına işe yaramadı. Sorunun kaynağı optimizer değil.

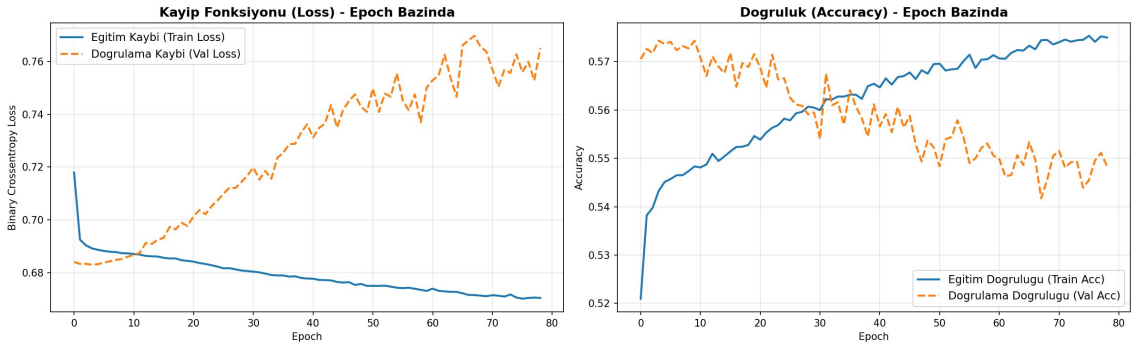
5. Deneme 5 — Büyük Model Denemesi (BAŞARISIZ)

Optimizer değişiklikleri sonuç vermeyince, bu sefer modelin mimarisini büyütmeyi denedik. Daha fazla katman ve daha fazla nöron, daha iyi öğrenme sağlar mı?

Kullanılan Ayarlar

Parametre	Değer
Mimari	Dense(128) → BN → Dense(64) → BN → Dense(32) → Dense(1)
Dropout	0.2 (0.5'ten düşürüldü)
Aktivasyon	swish (ReLU yerine)
BatchNormalization	Her Dense'den sonra
Optimizer	AdamW(learning_rate=0.001, weight_decay=0.0001)
Batch Size / Epoch	128 / 200
Callbacks	EarlyStopping + ReduceLROnPlateau
Toplam Parametre	12.417 (önceden 2.753 idi — 4.5 kat arttı)

Eğitim Grafiği (Uzun Koşu - 80 epoch)



Bu grafik 'kitap resmi' overfitting örneğidir. Train loss düşerken Val loss tırmanıyor.

Sonuçlar

Metrik	Değer
Train Loss	0.6704
Val Loss	0.7651 (korkunç — artıyor!)
Train-Val Fark	0.0947 (çok büyük)
Test Loss	0.6911
Test Accuracy	%55.45 (en düşük)
Eğitim Epoch	79

Yorumlama — Bu Neden Başarısız Oldu?

Grafikte görülen durum klasik 'overfitting' örneğidir: Train ve Val eğrileri 10. epoch civarında kesişiyor ve sonra birbirlerinden ayrılıyorlar. Makas açılıyor. Bu 'ezberleme'nin görsel imzasıdır.

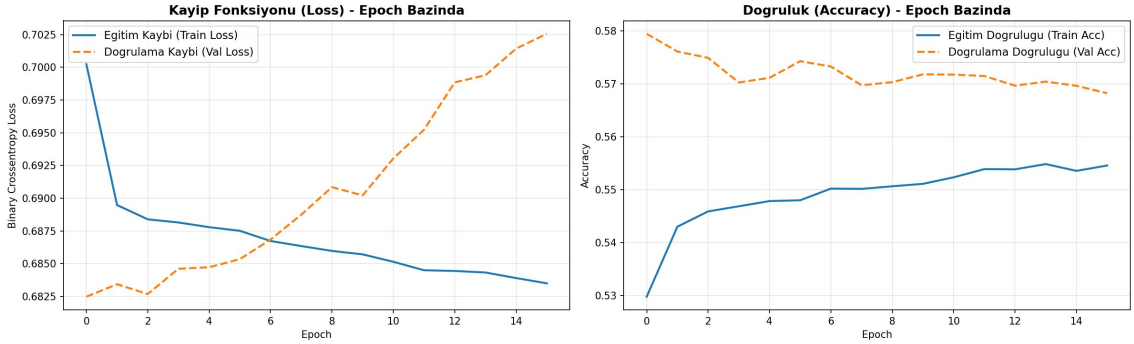
Neden 12.417 parametre çok fazla? Burada bir yanılgıyı düzeltelim: Veri setinde 236.000 satır olması büyük model kullanmaya yetmez. Önemli olan bilgi yoğunluğudur. Finansal veride her satırın büyük kısmı gürültüdür — gerçek sinyal çok azdır. Küçük modelin sınırlı parametreleri sadece 'güçlü' kalıpları öğrenebilir (ki zaten verinin sunabileceği de bu kadar). Büyük modelin bolca fazla parametresi var — bu fazlalık, rastgele gürültüyü 'kalıpmış gibi' öğreniyor. Test setinde bu sahte kalıplar tekrarlanmadığı için accuracy düşüyor.

Durum: BAŞARISIZ. Büyük model küçük modelden daha kötü sonuç verdi.

6. Ara Denemeler — Farklı Konfigürasyonlar

Büyük modelin overfitting yaptığını fark ettikten sonra dengeli bir nokta bulmak için iki ara deneme yapıldı. Her ikisi de başarısız oldu ama farklı sebeplerle.

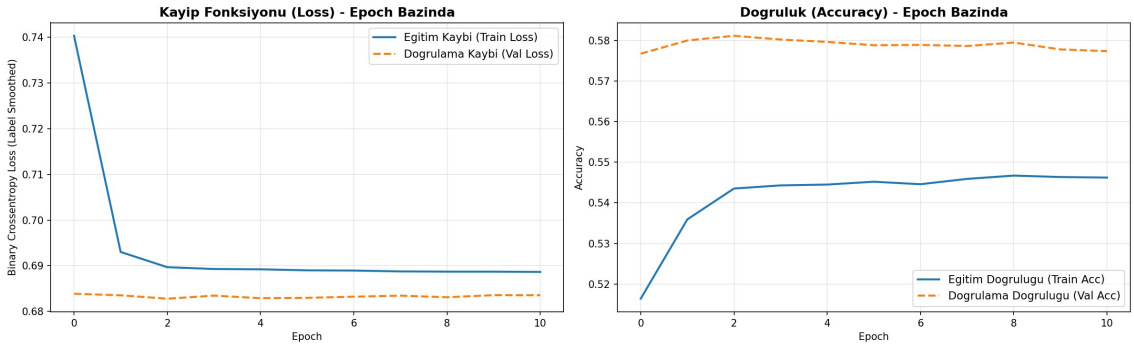
Ara Deneme 1 — Aynı büyük model, erken duruş (16 epoch)



Sonuç: Train Loss 0.6835, Val Loss 0.7026, Fark 0.0191. Val loss daha 10. epoch'ta yukarı dönmeye başlamış.

Yorum: Bu da overfitting ama daha erken yakalandı. EarlyStopping aktif olduğu için modelin en iyi ağırlıkları (epoch 2-3 civarı) korundu. Test sonucu yine de iyileşmedi çünkü sorun modelin kapasitesinde.

Ara Deneme 2 — Daha fazla regularizasyon (v3: Dropout 0.35 + Label Smoothing)



Sonuç: Val loss tamamen düz bir çizgi halinde — model hiçbir şey öğrenmiyor.

Yorum: Bu seferki tersine — regularizasyon aşırı oldu. Model eğitimin ilk 2 epoch'undan sonra hiçbir şey öğrenememe noktasına geldi. Grafikte val loss düz bir çizgi çünkü her tahmin için aynı olasılık üretiyor. Bu 'donmuş' bir modeldir.

Önemli ders: Regularizasyon bir denge meselesidir. Az olursa ezber (overfitting), fazla olursa donma (underfitting). Finansal verinin kendine özgü zor yanı şudur: sinyal gürültüden çok küçük. Bu yüzden ideal denge penceresi çok dar.

Durum: Her iki varyasyon da başarısız — model büyüklüğü bu problemin çözümü değil.

7. Deneme 6 — Yeni Veri Seti (EN İYİ SONUÇ)

Mimari denemelerinin hepsi aynı noktaya (%55-56 accuracy) çarptıktan sonra, yaklaşımı değiştirip farklı bir veri seti denedik. Bu sefer değişen model değil, veri oldu.

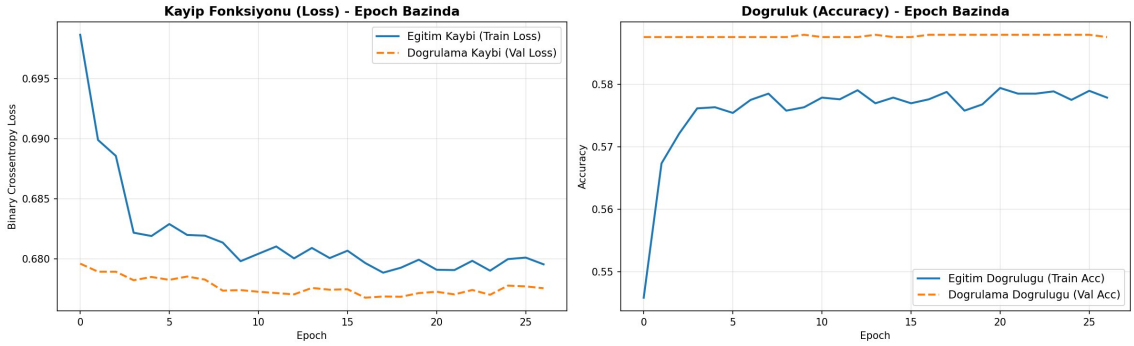
Yeni Veri Setinin Farkları

Özellik	Eski Veri Seti	Yeni Veri Seti
Satır Sayısı	~236.000	~17.500 (13 kat az)
Özellik Sayısı	9 (MACD, Bollinger)	7 (Hacim ve SMA)
Teknik gösterge tipi	MACD + Bollinger	Hacim bilgisi + SMA uzaklığı
Yeni özellikler	—	Log_Hacim, Hacim_Degisimi

Kullanılan Ayarlar

Parametre	Değer
Mimari	Dense(64) → Dropout(0.5) → Dense(32) → Dropout(0.5) → Dense(1)
Aktivasyon	ReLU / Sigmoid (Deneme 1 ile aynı)
Optimizer	Adam(learning_rate=0.001)
Batch Size / Epoch / Patience	64 / 50 / 10
Toplam Parametre	2.625
Yeni ek	restore_best_weights=True

Eğitim Grafiği



En sağlıklı eğitim grafiği. İki eğri birlikte düşüyor, aralarında hiç makas yok.

Sonuçlar

Metrik	Değer
Train Loss	0.6795
Val Loss	0.6775
Train-Val Fark	0.0020 (en küçük)
Test Loss	0.6838
Test Accuracy	%56.77 (en yüksek)

Random Baseline Loss	0.6839
Edge (baseline'a göre kazanım)	+0.0001 (küçük ama gerçek)
Eğitim Epoch	27

Yorumlama — Bu Neden Başarılı Oldu?

Paradoks: 13 kat daha az veriyle daha iyi sonuç alındı. Bu bize şunu öğretir: satır sayısı bilgi miktarını belirlemez, özellik kalitesi belirler.

Neden başarılı? İki sebebi var:

1) Hacim bilgisi çok değerli: Yeni veri setinde Log_Hacim ve Hacim_Degisimi özellikleri var. Piyasa mikroyapısında hacim, fiyat yönünün önemli bir öngörücüsüdür. 'Yüksek hacimle yükseliş güçlü bir trendin başlangıcı olabilir' tarzı bilgiler modelde yeni sinyaller yaratıyor. MACD ve Bollinger zaten fiyattan türetilen göstergelerdi — yeni bilgi eklemiyorlardı.

2) Veri seti daha temiz ve tutarlı: Sadece 11 hisse ve 5 olay içeriyor. Eski veri setinde çok fazla farklı hisse, farklı olay tipi ve farklı rejim karışıktı. Yeni veri daha dar bir alanda daha tutarlı kalıplar içeriyor — model bunu öğrenebiliyor.

Grafiğin kalitesi: Train ve Val eğrileri neredeyse üst üste, aralarında 0.0020 fark var. Bu, modelin ezberlemediğini ve veriden gerçek kalıpları öğrendiğini gösterir. İdeal bir eğitim grafiği budur.

Durum: BAŞARILI. Tüm denemeler arasında en iyi sonuç. Test Acc %56.77.

8. Tüm Denemelerin Karşılaştırmalı Özeti

Deneme	Optimizer / Model	Test Loss	Test Acc	Durum
Deneme 1	Adam, küçük model	—	—	Baseline
Deneme 2	Adam lr=0.0001, büyük batch	0.6865	%55.87	Kabul edilebilir
Deneme 3	AdamW varsayılan	0.6868	%55.93	Kabul edilebilir
Deneme 4	AdamW düşük LR, uzun eğitim	0.6867	%55.77	Kabul edilebilir
Deneme 5	Büyük model (12k param)	0.6911	%55.45	BAŞARISIZ (overfitting)
Deneme 6	Yeni veri seti, küçük model	0.6838	%56.77	EN İYİ

Neden Bazı Denemeler Başarısız Oldu?

Deneme 5'in (büyük model) başarısızlık sebebi: Model parametresi (12.417) finansal verinin içerdiği efektif sinyale göre çok fazlaydı. Parametre sayısının artması, veriden daha fazla kalıp çıkarma yeteneği demektir. Ama finansal veride 'çıkarılacak kalıp' sınırlı olduğu için, fazla parametre gürültüyü öğrenmeye başladı. Bu da klasik overfitting demektir.

Ara deneme 2 (v3) başarısızlık sebebi: Bu sefer tam tersi — regularizasyon aşırı sıkı olunca model eğitim yapamadı, donup kaldı. Underfitting.

Neden Deneme 2/3/4 benzer sonuçlar verdi? Hepsinde aynı küçük model mimarisi kullanıldı. Bu mimari verideki gerçek sinyali zaten yakalıyordu. Optimizer değiştirmek veya learning rate ile oynamak, bu sinyal tavanını aşamadı. Bu çok önemli bir bulgudur: sorun modelin ayarlarında değil, verideki sinyalin doğal sınırlarında.

Neden Deneme 6 Başarılı Oldu?

Kısacası: daha iyi özellikler (hacim bilgisi) + daha temiz veri seti. Model mimarisi Deneme 1 ile aynı. Fark tamamen veride. Bu, makine öğrenmesinin bilinen bir kuralını doğruluyor: 'Garbage in, garbage out' (çöp girer, çöp çıkar). Modelin kapasitesinden çok önce, verinin kalitesi önemlidir.

9. Çıkarılan Dersler

Ders 1 — Küçük Model Her Zaman Daha Kötü Değildir

Büyük modeller daha iyi öğrenir sanısı yaygındır, ama bu güçlü sinyalli problemler için geçerlidir (örneğin görüntü tanıma). Finansal tahminde veri gürültülüdür, sinyal zayıftır. Küçük bir model (2.500-3.000 parametre) bu tür verilerde büyük modelden daha iyi iş çıkarır. Sebebi: fazla parametre = fazla ezberleme kapasitesi = fazla overfitting.

Ders 2 — Hiperparametre Ayarlamanın Sınırı Vardır

Denemeler 2, 3 ve 4'te optimizer, learning rate, batch size, epoch, patience hepsi değiştirildi. Sonuç hep %55.8 civarı kaldı. Bu çok önemli bir gerçeği ortaya koyuyor: hiperparametre ayarının kazanabileceği maksimum yaklaşık %1-2 accuracy'dir. Bundan fazlasını istiyorsanız veri veya problem tanımı düzeyinde değişiklik gerekir.

Ders 3 — Veri Kalitesi Model Boyutundan Önemlidir

Aynı model mimarisi iki farklı veri setinde test edildi: büyük olan %55.87, küçük olan %56.77. Satır sayısı 13 kat azaldı ama sonuç iyileşti. Sebep: yeni veri setindeki özellikler (hacim bilgisi) daha bilgi içerikliydi. Özellik mühendisliği, model mühendisliğinden daha değerlidir.

Ders 4 — Grafik Okuması Önemlidir

Sadece son sayılara bakmak yanıltıcıdır. Bir grafiğe baktığımızda şunları kontrol etmeliyiz:

- Train ve Val eğrileri birlikte mi gidiyor? Evet ise sağlıklı.
- Makas açılıyor mu? (Train düşerken Val yükseliyor) Evet ise overfitting.
- Her ikisi de düz mü? Donmuş model, underfitting.
- Val loss ne zaman minimum? EarlyStopping doğru yerde durdurdu mu?

Ders 5 — Finansal Tahminin Gerçek Limitleri

Literatürde S&P; 500 tarzı endekslerin günlük/haftalık fiyat yönü tahmininde normal accuracy %54-58 aralığıdır. %60 üstü çok nadirdir ve genelde şüpheyle karşılanır (veri sızıntısı olabilir). Bizim %56.77 sonucumuz bu aralıkta ve literatürle tutarlıdır. Bu bir başarısızlık değil, normal finansal veri karakteristiğidir.

Nihai Değerlendirme

Bu çalışma, derin öğrenme ile finansal tahmin yapmanın zorluklarını ve çözüm yaklaşımlarını göstermektedir. 6 deneme boyunca şu sonuçlar elde edildi:

- En iyi mimari: Dense(64) → Dropout(0.5) → Dense(32) → Dropout(0.5) → Dense(1)
- En iyi optimizer: Adam (varsayılan) — farklı varyantlar çok az fark yarattı
- En iyi veri seti: 17.5k satırlık hacim bilgili veri (Deneme 6)
- En iyi test accuracy: %56.77
- Kaçınılması gereken: Büyük modeller (12k+ parametre) bu verilerde overfitting yapar

Sonuç: Finansal tahminde model değişiklikleri sınırlı kazanım verir. Asıl kazanım özellik mühendisliğinden ve veri kalitesinden gelir. %56.77 accuracy, rastgele tahminin üzerinde gerçek bir sinyal olduğunu göstermektedir.